Exploring Cold App Launch Behavior on Android using eBPF

An eBPF-Guided Study of Android App Launch Behavior and Cold-Start Optimization

CS7601 – Operating Systems
Spring 2026 – Final Project Report

Fathy Ibrahim

April 30, 2026

Abstract

This report presents an end-to-end study of file I/O, page allocation, and allocation latency on an Android device using eBPF-based kernel observability, followed by an exploration of whether eBPF can help reduce repeated cold-launch overhead for Android Mobile Applicaton (the AOSP Gallery app is taken as a case study). The first half of the report covers three characterization tasks that establish the methodological foundation: classifying file access patterns, measuring page allocation order distributions, and measuring page allocation latency under different workloads. The second half of the project shifts from measurement to exploration by focusing on one practical mobile systems question: can repeated cold launch time be improved on Android with small, realistic interventions? To study that, the project compares four lightweight optimization attempts, two in user space and two in the kernel, against a controlled cold-launch baseline. The results show that some simple ideas have little real impact, and some can even be misleading if they mostly change what is being observed rather than what the system is actually doing. At the same time, one targeted kernel-guided scheduling approach produces a clear reduction in repeated cold-launch latency.

Contents

1 Introduction **3**

2 Part I – Characterization **3**
2.1 Task 1 – File I/O Pattern Analysis . . . 3
2.1.1 Overview . . . 3
2.1.2 Initial Design Challenge . . . 3
2.1.3 Dual-Hook Design with State Tracking . . . 3
2.1.4 Experimental Setup . . . 4
2.1.5 Results and Analysis . . . 4
2.2 Task 2 – Page Allocation Order Analysis . . . 6
2.2.1 Overview . . . 6
2.2.2 Initial Issue and Fix . . . 6
2.2.3 Experimental Setup . . . 6
2.2.4 Results and Analysis . . . 6
2.3 Comparison Between Tasks 1 and 2 and the Move to Task 3 . . . 8
2.4 Task 3 – Page Allocation Latency Analysis . . . 9
2.4.1 Overview . . . 9
2.4.2 Design Challenge and Dual-Probe Solution . . . 9
2.4.3 Experimental Setup . . . 9
2.4.4 Results and Analysis . . . 9

3 Part II – Exploration **11**
3.1 Exploration Promise 11
3.2 Exploration Setup and Core Assumptions 11
3.3 Baseline 12
3.4 First Experiment: Simple User-Space Warm-Up (Failed) 14
3.5 Second Userspace Warm-Up 15
3.6 Third Experiment: Kernel-Side Selective Startup Focus (Failed) 16
3.7 Fourth Experiment: Kernel-Guided Launch Priority Boost 17

4 Conclusion **19**

A Appendix: Build and Run Commands **19**

1 Introduction

Modern Android devices expose a rich and increasingly observable kernel surface, and extended Berkeley Packet Filter (eBPF) provides a safe, programmable mechanism for attaching probes to that surface without modifying the kernel source tree. This project uses eBPF as the unifying instrumentation framework to study three aspects of system behavior on a real Android device: file I/O patterns, page allocation sizes, and page allocation latency. These three dimensions correspond to fundamental operating-system primitives – the storage stack, the memory allocator, and the timing of allocator operations – and together they describe a substantial portion of the system activity that takes place during user interactions.

The project is organized in two halves. The first half consists of three *characterization tasks*, each of which builds an eBPF-based monitoring pipeline, defines a controlled experimental setup, and analyzes the resulting measurements. The second half consists of an *exploration* that uses the characterization findings to ask a focused question: can eBPF-guided observation help reduce repeated cold-launch overhead for an Android application that is sensitive to storage and memory activity? The AOSP Gallery application is used as the target throughout the exploration. To make the comparison reproducible and fair, every experiment in both halves of the project is repeated five times (four runs for the characterization experiments and five runs for the exploration experiments) and reported as averages together with per-run values, so that results are not dominated by single-run noise.

2 Part I – Characterization

The characterization half of the project consists of three tasks that progressively build the eBPF measurement infrastructure used later in the exploration. Each task focuses on a single dimension of kernel behavior, defines its own classification or metric, and runs a controlled set of workloads. All characterization experiments were run **four times per workload** and reported as averages, in order to reduce single-run variability and improve reproducibility.

2.1 Task 1 – File I/O Pattern Analysis

2.1.1 Overview

The first task developed an eBPF-based monitoring system to analyze the file I/O behavior of foreground applications on an Android device. The objective was to classify each file access into one of three categories: *sequential I/O*, *random I/O*, or *unclassified I/O*. The system has two components: a kernel-side eBPF program attached to the `raw_syscalls/sys_enter` and `raw_syscalls/sys_exit` tracepoints, and a user-space monitor that collects and aggregates per-syscall statistics. The goal was to understand how different user actions affect the volume of read and write operations, the shape of the access pattern, and the visibility of user behavior at the kernel level.

2.1.2 Initial Design Challenge

An early version of the monitor only attached to `sys_enter`. This was quickly shown to be insufficient: the standard `read` and `write` syscalls do not carry an explicit file offset in their arguments, and only positional variants such as `pread64` and `pwrite64` provide one directly. As a result, classification coverage was extremely low; in one representative measurement, only 4 of 2,246 reads could be classified, which made the resulting statistics effectively meaningless.

2.1.3 Dual-Hook Design with State Tracking

To address this, the system was redesigned around a dual-hook strategy. The `sys_enter` hook captures the syscall type and its parameters, while the `sys_exit` hook captures the return value, which corresponds to the actual number of bytes read or written. To recover offsets for

non-positional syscalls, the eBPF program maintains per-(PID, file descriptor) state that records the previous offset and previous transfer size. The current offset for a non-positional read or write is then inferred from the previous state. Classification follows the simple rule that an access is sequential when its starting offset equals the sum of the previous offset and the previous size, and random otherwise. An *unclassified* category was retained to handle incomplete cases, for example when no prior state exists for a descriptor. With this redesign, classification coverage rose to approximately 95–100%, and both positional and non-positional I/O could be handled uniformly.

2.1.4 Experimental Setup

All Task 1 experiments share the same conditions: short observation windows, identical device and environment, the same eBPF pipeline, foreground-process filtering, and four runs per experiment. Three workloads were chosen to expose qualitatively different I/O patterns:

Experiment A – Web Browsing (Image Scrolling). Continuous scrolling on Google Images, with frequent loading of new content. The purpose was to observe a dynamic, mixed I/O workload.

Experiment B – File Copy (Large Video Transfer). Copying a large (approximately 5.68 GB) video file as a continuous read–write loop, in order to observe a pure sequential workload.

Experiment C – Video Seek (Timestamp Jumping). Jumping repeatedly between timestamps in a playing video to attempt to observe random access patterns from the user perspective.

2.1.5 Results and Analysis

I/O volume. Web browsing produced the highest I/O volume due to many small operations driven by content fetching. The file-copy workload produced a moderate volume because large buffers reduce the number of syscalls per byte transferred. Video seeking produced a moderate volume that depended strongly on the underlying buffering behavior of the media stack.

Sequential vs. random distribution. Across all three workloads, the global pattern observed at the kernel level was strongly dominated by sequential I/O, with random I/O appearing only as a small fraction at most. Web browsing was mostly sequential with a small fraction of random accesses, consistent with chunk-based downloads, cached reads, and append-style writes. The file-copy workload was effectively 100% sequential, as expected from a linear traversal of the source file. The video seek workload was the most surprising: although the user behavior was clearly random in nature, the kernel-level view was almost entirely sequential.

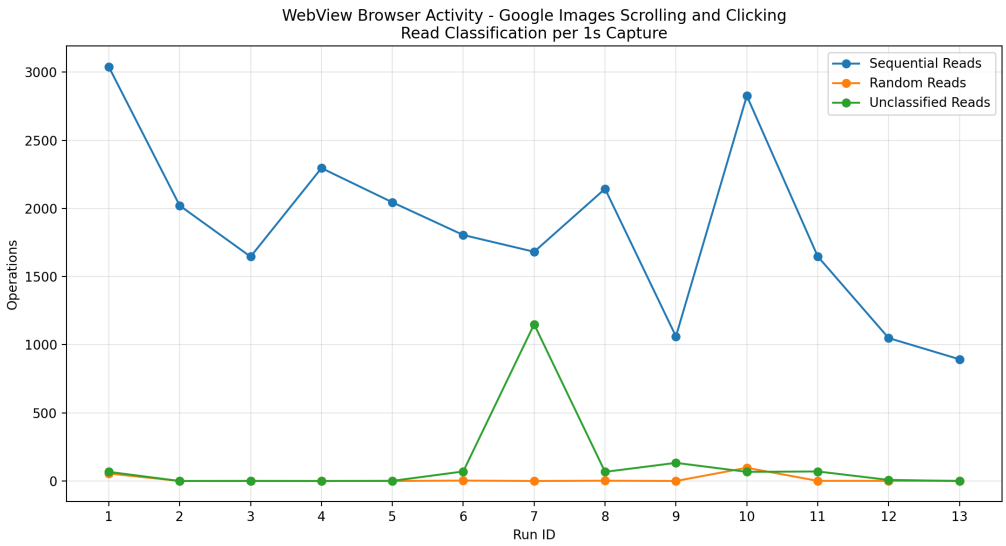

Figure 1: Web Browsing activity: per-run distribution of sequential, random, and unclassified file I/O operations.

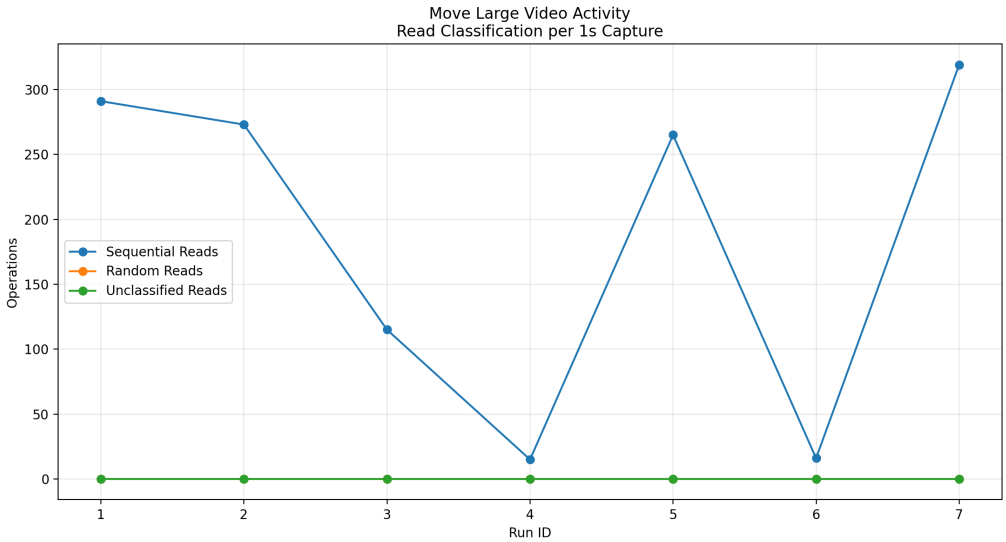

Figure 2: Move Large Video activity: per-run classification of read operations into sequential, random, and unclassified categories across runs.

Key insight: kernel view $\neq$ user behavior. The video-seek experiment exposes a fundamental observation. The foreground application typically does not issue the random reads itself; instead, media services such as `mediaserver`, `mediaextractor`, and the codecs perform the actual file access. These services may handle seeks internally and expose a processed, sequential stream to the foreground app. Consequently, what appears as random behavior to the user looks sequential at the foreground process boundary. Monitoring only the foreground process therefore does not capture the full system behavior and may hide underlying random access patterns.

Variability. Web browsing exhibited high run-to-run variability driven by user interaction, content loading, and caching. The file-copy workload was very stable. The video seek workload was stable but misleading, because the apparent regularity at the kernel level masks the system-level complexity introduced by media-stack abstractions.

Conclusion. At the kernel level, most observed file I/O appears sequential, even for complex user interactions. Random I/O is rare and appears mainly during transitions. Android's architecture introduces an abstraction layer in which much of the actual storage activity is delegated to background services, so the foreground-process view is simplified and sometimes misleading. This mismatch between user-visible behavior and kernel-visible behavior is one of the most important lessons of Task 1 and motivates the careful, kernel-informed approach used later in the exploration.

2.2 Task 2 – Page Allocation Order Analysis

2.2.1 Overview

Task 2 implemented an eBPF-based monitoring system for page allocation behavior in the Linux kernel on the same Android device. The kernel-side program attaches to the `kmem/mm_page_alloc` tracepoint and collects the *order* of each allocation – that is, the size of the allocation expressed as a power of two. The goal was to characterize how different workloads influence both the total number of allocations and the distribution of allocation sizes.

2.2.2 Initial Issue and Fix

An early measurement issue was that the eBPF map used to store allocation counts was pinned and persisted across runs. As a result, allocation counts accumulated across runs and produced inflated values that did not reflect the actual workload. Repeating the same experiment without any new activity nonetheless produced increasing counts. To resolve this, the user-space monitor was modified to clear the pinned map at the start of every run, so that each three-second sampling window starts from a known zero state and runs are independent and comparable.

2.2.3 Experimental Setup

All Task 2 experiments use a three-second sampling window, the same device and environment, the same eBPF pipeline, and four runs per workload. Three workloads were chosen:

Idle Screen On. The screen is on but there is no user interaction; only background system processes are active. This establishes a baseline for memory activity.

Application Launch Burst. Several applications (Contacts, Gallery, Camera) are launched in rapid succession inside the three-second window, capturing transient memory pressure during initialization.

Image Scrolling. Continuous fast scrolling through Google Images, generating sustained decoding and rendering activity for the full sampling window.

2.2.4 Results and Analysis

Total allocation volume. The relationship across workloads is clear: idle activity produces a very small number of allocations, application launches produce a substantial burst, and sustained image scrolling produces the largest sustained allocation rate. The average totals over the three-second window were approximately a few tens of allocations under idle conditions, around 35,000–47,000 for the launch burst, and around 57,000–102,000 for image scrolling, consistent with the visible gap between the three workloads.

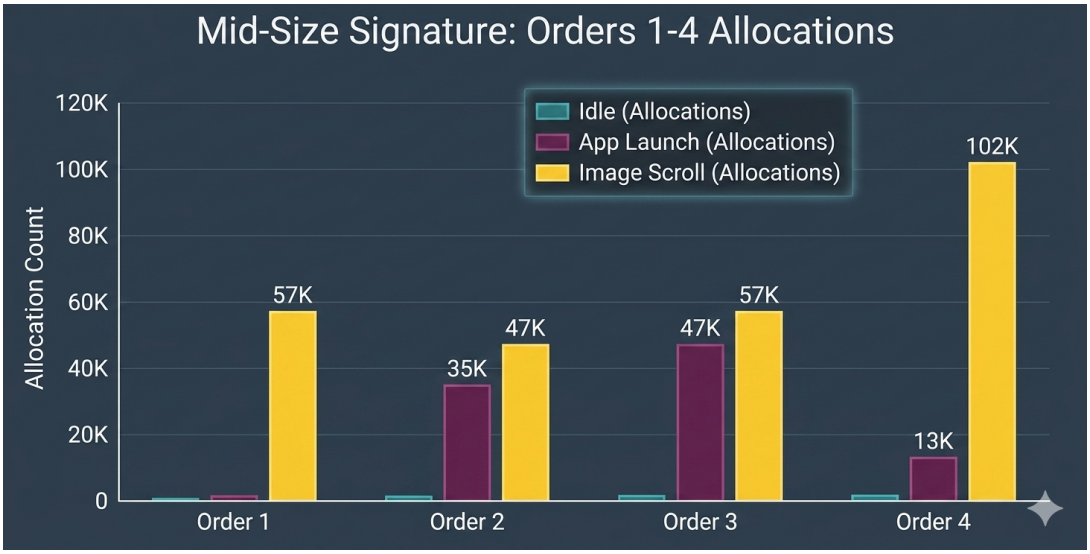

Figure 3: Illustration of the 99% allocation rule and mid-size allocation signature. Most allocations occur at order-0, while mid-sized allocations (orders 1–4) exhibit distinct workload-dependent patterns, including application launch and interactive workloads such as image scrolling.

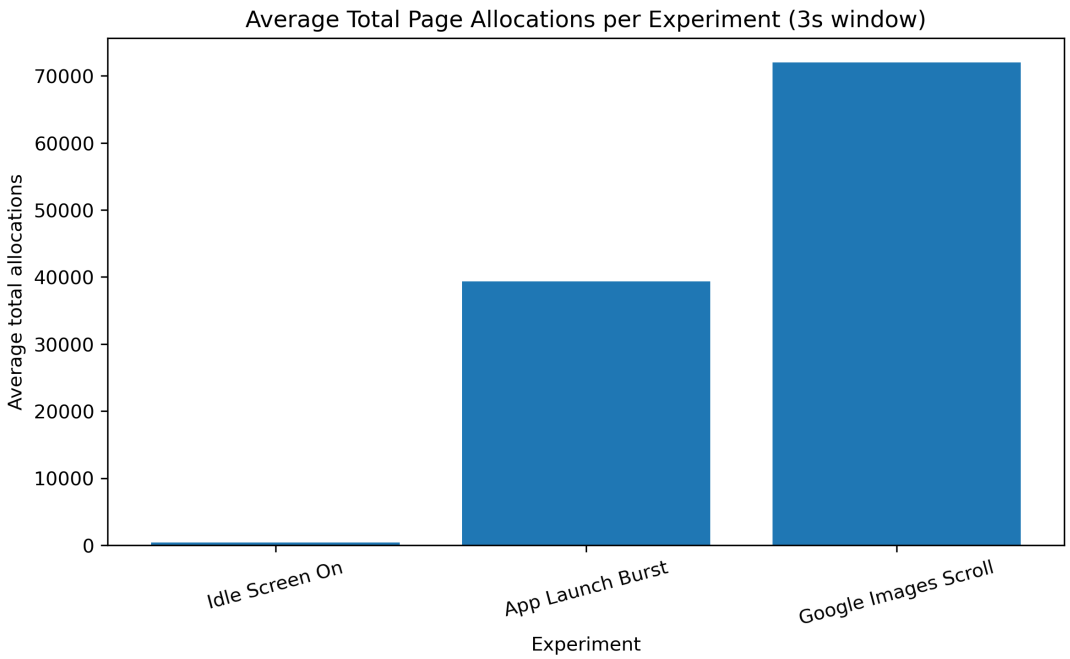

Figure 4: Average total page allocations per workload, illustrating differences in allocation intensity across the three workloads.

Allocation order distribution. Across all three workloads, order-0 allocations (single 4 KB pages) account for approximately 99% of all allocations, confirming that the kernel strongly favors fine-grained allocations. Idle activity is essentially entirely order-0, with a very small share of order-2 and order-3 allocations. Application launch shows a small but non-trivial share of orders 1–3 and occasional order-4 and even order-9 allocations, reflecting the larger contiguous allocations occasionally needed during initialization. Image scrolling shows a consistent share of orders 1–3 and occasional order-4 allocations, consistent with image decoding buffers and

rendering pipeline structures.

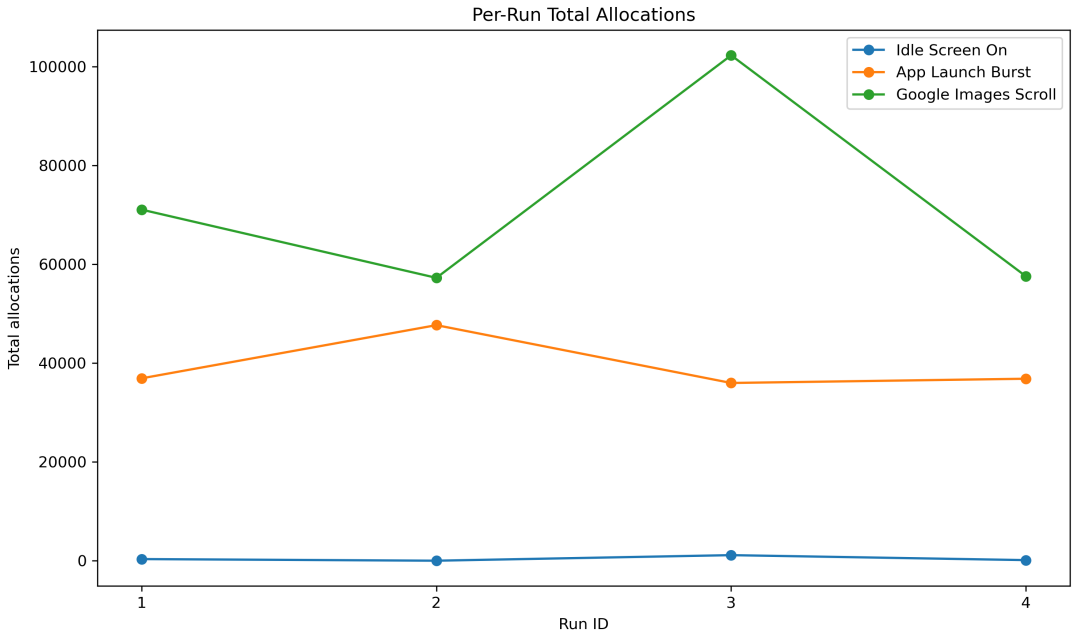

Figure 5: Average normalized allocation distribution by order (%) across the three workloads.

Variability. Idle conditions show high relative variability, since the absolute counts are small and dominated by background services and scheduling. The application launch burst is comparatively stable. Image scrolling is more variable in absolute terms because scroll speed, network loading, and image sizes all influence the instantaneous allocation rate.

Conclusion. Page allocation on this device is overwhelmingly dominated by order-0 allocations, indicating that most memory requests are small and fine-grained. Workload intensity strongly affects total allocation volume, and higher-order allocations become measurable mainly under continuous, decoding-heavy workloads. The kernel's preference for small allocations is consistent with its strategy of reducing fragmentation and improving allocator efficiency, with larger contiguous allocations reserved for cases that genuinely need them.

2.3 Comparison Between Tasks 1 and 2 and the Move to Task 3

Task 2 was, by design, a stateless counting exercise: a single tracepoint (`kmem/mm_page_alloc`) was sufficient, the eBPF program incremented an order-indexed histogram map, and the monitor only needed to clear and read that map between runs. Each allocation was treated independently and immediately aggregated. Task 3 introduces a substantially different design because it requires *stateful timing*. Measuring latency requires two hook points – a kprobe on `__alloc_pages` for the start timestamp and the existing tracepoint for the completion event – and an additional pinned map (`inflight_map`) that temporarily stores per-thread start times keyed by `pid_tgid` so that events can be correlated. The histogram map is replaced with a richer `latency_map` that aggregates count, total latency, and minimum and maximum latency per order. The monitor still clears and reads the aggregated map, but it now also computes averages from these aggregated counters.

2.4 Task 3 – Page Allocation Latency Analysis

2.4.1 Overview

Task 3 extends the eBPF-based monitoring system to measure *how long* each page allocation takes, while still tracking the allocation order. The kernel-side program uses a kprobe on `__alloc_pages` to capture the allocation start and the `kmem/mm_page_alloc` tracepoint to capture the allocation end. The user-space monitor aggregates latency statistics per order. The goal is to characterize the relationship between workload intensity, allocation frequency, and per-allocation latency.

2.4.2 Design Challenge and Dual-Probe Solution

A single tracepoint cannot measure latency, because the tracepoint fires after the allocation has completed and provides no timing information. A two-point measurement is therefore required: a start event captured by a kprobe on `__alloc_pages`, and an end event captured by the existing tracepoint. The eBPF program correlates the two events through an in-flight map keyed by `pid_tgid` that stores the start timestamp and the requested order. When the allocation completes, the program looks up the start record, computes the latency, and updates per-order aggregate counters (count, total latency, min, max). Aggregating in the kernel rather than streaming each event keeps overhead low and matches the design style established in Task 2.

2.4.3 Experimental Setup

Three workloads were chosen, each repeated four times, with a three-second sampling window per run.

Screen-Off Idle. The screen is off and there is no user interaction. This is intended to capture cold allocator paths and establish a high-latency, low-frequency baseline.

Screen-On Static Page. The screen is on with one webpage opened, with no interaction during the measurement. This represents a moderate steady-state workload.

Web Scrolling. Continuous scrolling through web content for the full three seconds, producing a high-frequency allocation workload that stresses the allocator's hot paths.

2.4.4 Results and Analysis

Average latency by workload. The observed averages displayed a clear and somewhat counterintuitive ordering: the screen-off workload had the highest average latency (roughly 25–55 μs), followed by the static page (roughly 7–12 μs), with scrolling exhibiting the lowest average latency (roughly 2–4 μs). In other words, the heavier the workload, the lower the per-allocation latency on average.

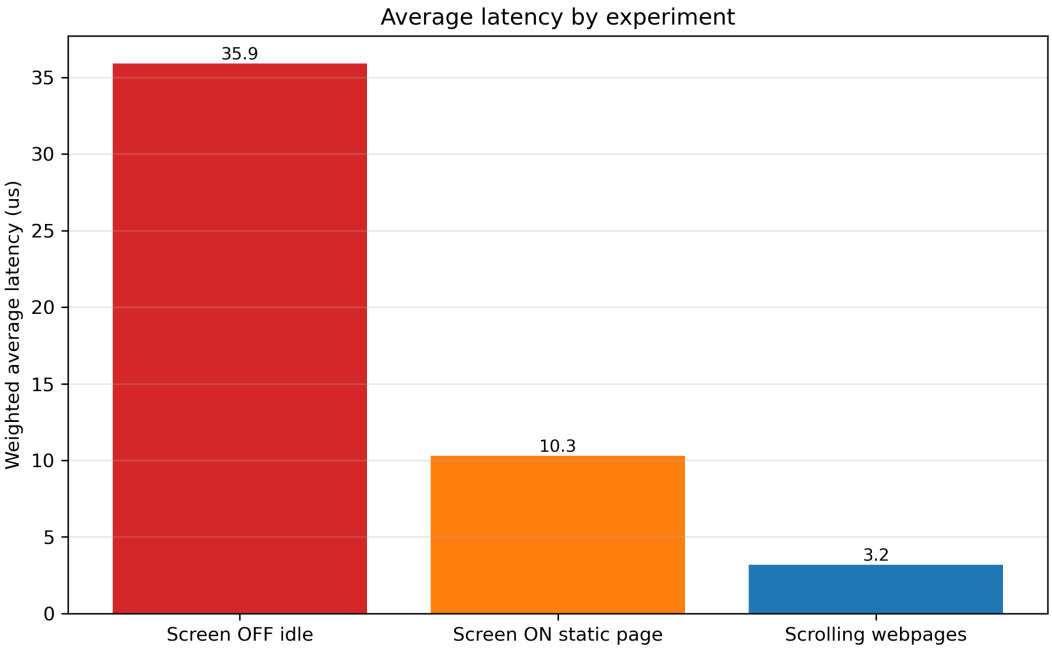

Figure 6: Average normalized allocation distribution by order (%) across the three workloads.

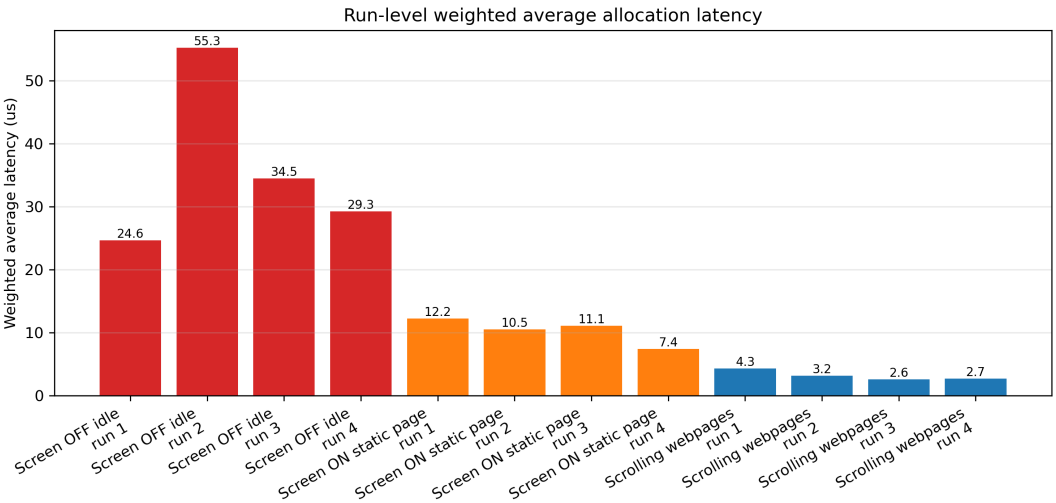

Figure 7: Run-level weighted average allocation latency, computed per run and per workload.

Allocation volume vs. latency. The corresponding allocation counts inversely mirror the latency ordering: a few to a few tens of allocations per window in screen-off, around 50–140 in the static page case, and roughly 11,000–59,000 during scrolling. The data therefore show a strong inverse relationship between allocation frequency and average latency.

Key insight: warm fast paths. Higher workloads keep allocator fast paths warm, keep relevant caches active, and allow the allocator to reuse internal state, all of which reduce per-call latency. Idle conditions, by contrast, are dominated by cold paths and infrequent allocations whose latency reflects reclaim or cache misses.

Latency by order. Order-0 allocations dominate every workload and exhibit the lowest latency. Higher orders (1–4) appear more often under heavier workloads and tend to incur higher latency, with greater variability. Scrolling introduces a noticeable share of orders 1–3 and occasional order-4 allocations whose costs reflect the higher search and fragmentation overhead at larger allocation sizes.

Worst-case latency. While average latency decreases under load, the maximum latency does not. Screen-off runs had maximum latencies up to about 1.5 ms, the static page case peaked around 80–160 μs, and scrolling produced occasional spikes up to about 4 ms. These spikes are consistent with occasional fallbacks to slower paths such as compaction or reclaim.

Conclusion. Page allocation latency is not constant; it depends strongly on workload intensity. Idle conditions yield infrequent but expensive allocations, moderate workloads yield balanced behavior, and heavy workloads yield frequent but on-average cheap allocations with occasional expensive spikes. Order-0 allocations dominate and are the fastest, while higher orders are slower and more sensitive to system state. The Linux allocator therefore performs better under load due to caching and hot allocation paths, but still exhibits occasional expensive operations.

3 Part II – Exploration

3.1 Exploration Promise

The characterization tasks established that the kernel-visible activity around file I/O, page allocation size, and page allocation latency is informative, controllable, and amenable to lightweight eBPF observation. The natural next question is whether those observations can be turned into something *actionable* – that is, whether eBPF-guided observability can help identify and reduce a concrete piece of overhead that real users experience. Cold application launches were chosen as the focal point of this second half of the project because they involve, simultaneously, all three of the dimensions already characterized: code and resource loading from storage, page allocation activity, and the latency of those allocations. They are also a familiar source of user-perceived friction, which makes any measurable reduction directly meaningful.

The promise of this exploration is intentionally modest. The aim is not to re-engineer the Android startup path, eliminate every contributor to launch latency, or claim a universal speed-up for arbitrary applications. The aim is specifically to ask whether eBPF-guided observation can help to recognize stable, repeated startup-related patterns for one well-chosen application, and whether a small, scoped intervention informed by those observations can shorten the storage-and-memory-related portion of repeated cold-launch overhead. To pursue this question honestly, several lightweight optimization techniques were explored, on both the user-space side and the kernel side. As the report will show, some of these techniques produced no meaningful improvement, while others produced measurable and defensible reductions in launch time. All attempts are presented, including those that failed, since the failures are themselves informative about what eBPF observation can and cannot do.

3.2 Exploration Setup and Core Assumptions

Target application. The target application throughout the exploration is the AOSP *Gallery* application. Gallery is a suitable target for a startup-focused study because its launch path is storage-sensitive, repeatable, user-visible, and largely independent of network conditions. Unlike cloud-backed or login-heavy applications, its cold-launch behavior is dominated by local file access and local initialization, which makes it a good fit for eBPF-based local observation and for controlled comparisons across runs.

Cold-launch definition. For the purposes of this work, a *cold launch* is defined operationally as launching Gallery after force-stopping it and confirming that its process is no longer running. This guarantees that the application must be re-created and re-initialized from the application's perspective, while keeping the procedure fully reproducible across runs. It does not, however, guarantee a fully cold system state in the strict sense – page caches, ART/Dex caches, profile data, and other persistent runtime artifacts can remain warm across runs and even across reboots. This distinction is important and is revisited later in the report when discussing the validity of warm-up baselines.

Measurement procedure. Every measured run follows the same protocol. First, Gallery is force-stopped and the device is returned to the home screen. Second, the BPF aggregation maps are prepared or cleared as required so that each run starts from a known zero state. Third, Gallery is launched, and startup is measured from the launch trigger until the first visible Gallery screen is presented. Fourth, kernel-visible startup activity – file activity, page allocation counts, and allocation latency statistics – is collected for the duration of the launch window. As in the characterization tasks, every exploration experiment is repeated five times and reported as both an average and a per-run sequence, so that conclusions are based on multiple observations rather than single measurements.

Role of the baseline. Before any optimization is evaluated, repeated cold launches are measured under the unoptimized configuration to establish a baseline. The baseline serves two purposes: it shows whether Gallery startup is itself stable enough to support the study – that is, whether launch time, startup-related file activity, and memory allocation behavior are repeatable across runs – and it provides the reference measurements against which all later optimization attempts are compared. Without this controlled reference, no claim about the effect of an intervention would be defensible.

Central exploration assumption. The central assumption of the exploration is that repeated cold launches of the same application exhibit stable startup-critical storage and memory patterns, and that a lightweight intervention targeting those patterns may reduce launch time. The four exploration experiments described later are different concrete ways of testing this assumption.

Scope and limitations. This work targets only the storage and memory-related portion of cold-launch overhead. Cold launch on Android is also affected by process creation, runtime initialization, class loading, resource setup, and application-specific startup logic, all of which are outside the scope of an eBPF-based observation methodology focused on file I/O and page allocation. Any improvement reported in the exploration should therefore be interpreted as a partial reduction in launch-related overhead rather than as a complete solution to application launch latency. The exploration is also limited to a single application on a single device and does not attempt to generalize beyond that configuration.

3.3 Baseline

The baseline establishes the unoptimized reference condition against which all later exploration attempts are compared. In this condition, Gallery is launched using the cold-launch procedure defined above, with *no optimization enabled in either user space or in the kernel.* There is no warm-up logic, no startup-file preloading, and no kernel-side selective tracing or scheduling modification; the system simply performs repeated cold launches under the definitions and assumptions already given. The baseline therefore serves as the experimental control: it provides the reference measurements for launch time, startup file activity, page allocation behavior, and (where relevant) allocation latency, and every later user-space and kernel-side exploration result must be interpreted relative to this baseline condition.

As in every other exploration experiment, the baseline was run **five times**, with averages reported alongside per-run values. The three figures that follow show the per-run baseline measurements: the number of distinct files touched during the launch window, the total number of page allocations during the launch window, and the cold-launch time for Gallery. Across the five runs, these quantities are stable enough to be used as a reference: file-touch counts cluster around an average of 18.2 files, total allocations average 5,708.6 allocations, and Gallery cold-launch time averages 203.4 ms.

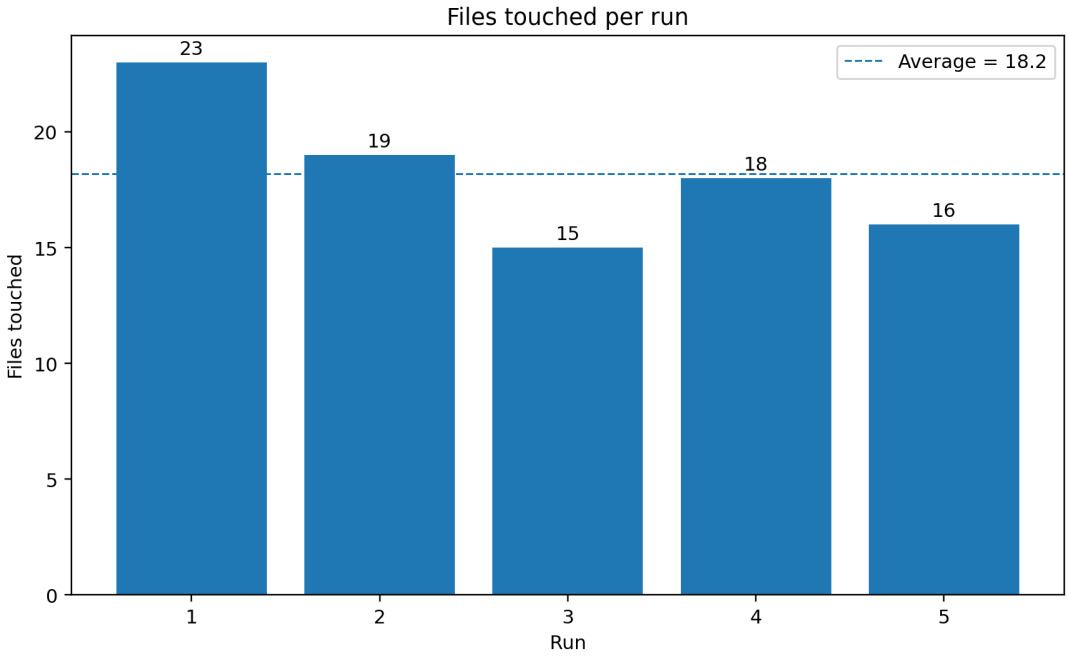

Figure 8: Baseline: number of distinct files touched per cold-launch run, with the five-run average.

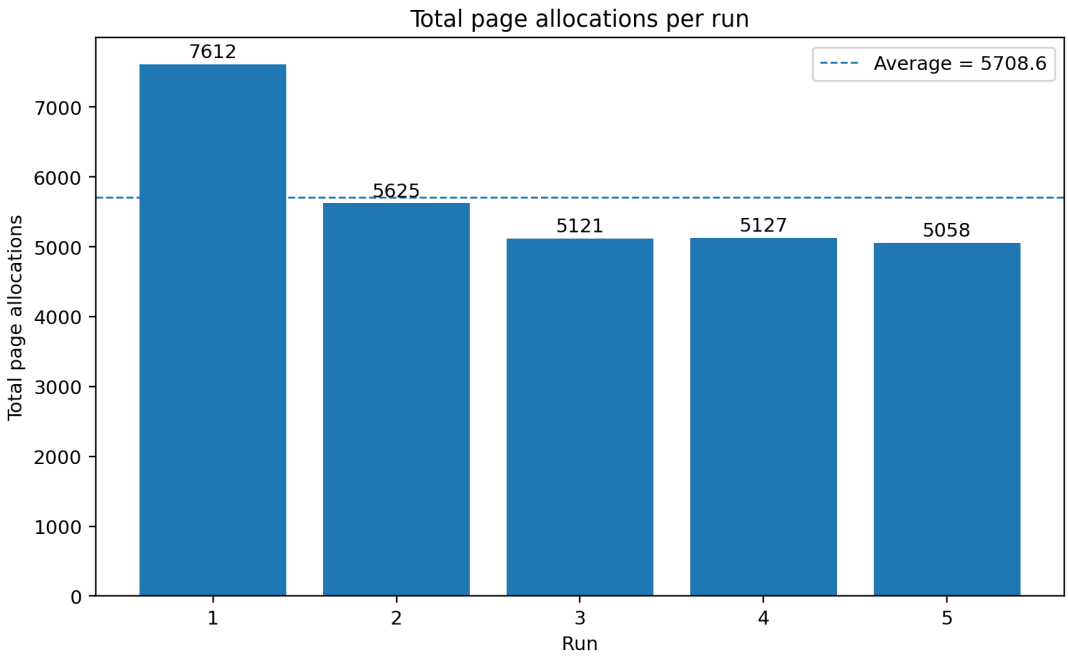

Figure 9: Baseline: total page allocations per cold-launch run, with the five-run average.

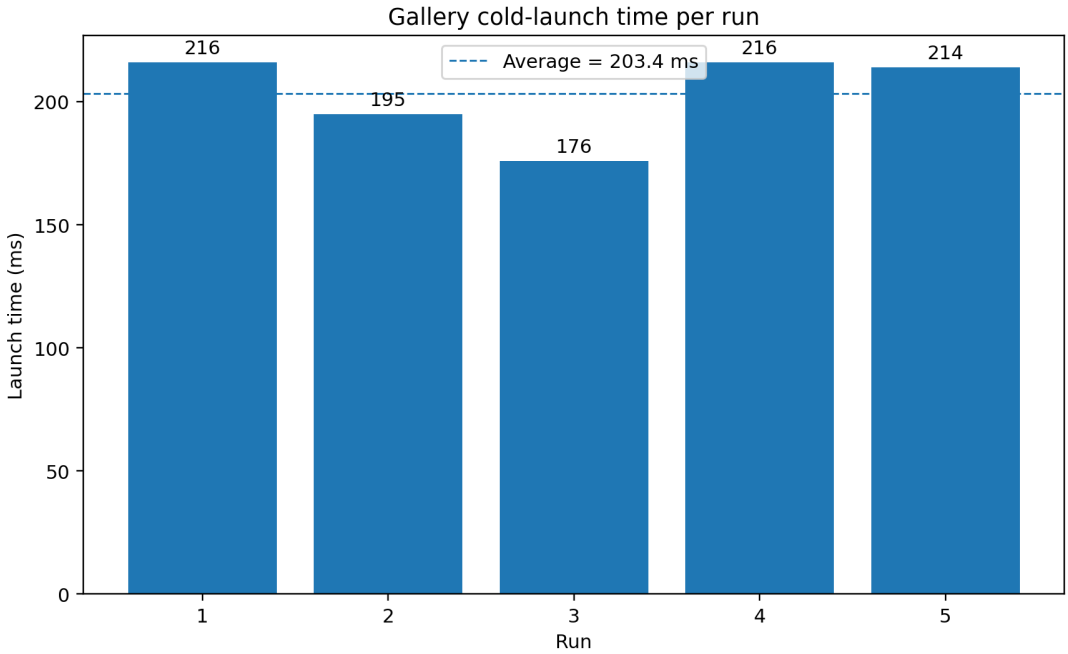

Figure 10: Baseline: Gallery cold-launch time per run, with the five-run average (≈ 203.4 ms).

The baseline measurements show that Gallery's repeated cold launches are sufficiently stable to support a controlled comparison: launch time variation across runs is modest, file-touch counts vary in a narrow band, and total page allocations cluster in a similar range. With this reference established, the remaining four subsections present the exploration experiments in order: two user-space attempts followed by two kernel-side attempts.

3.4 First Experiment: Simple User-Space Warm-Up (Failed)

The first exploration experiment is a deliberately simple, lightweight user-space warm-up implemented in `monitor_example.cpp`. The motivating idea is that running Gallery briefly before the measured launch may bring some startup-related state into a warmer condition, so that the subsequent measured launch can benefit from that state. The state being targeted is broad and indirect: page-cache contents corresponding to files touched during startup, possibly some startup-related framework or resource state, and any other ephemeral state attached to repeated access of the resources Gallery requires during launch.

The procedure for this experiment is conceptually straightforward. Gallery is launched once briefly, allowed to perform its startup-related file and resource access, and then killed. After this unmeasured pre-launch, Gallery is launched again and the second launch is the one that is timed and instrumented. The hope is that some of the state populated by the first launch persists through the brief delay and reduces the work that must be done during the measured launch. This is best described as a *lightweight user-space warm-up attempt* rather than a real optimization: it does not consult any kernel observation about which files matter, it does not attempt to preload any specific resource, and it makes no use of the eBPF instrumentation already in place. Honestly framed, the question being evaluated is simply: *can a simple pre-launch warm-up reduce repeated cold-launch overhead?*

The measured answer was effectively *no*. Repeated cold-launch measurements under this warm-up procedure did not produce a clear or stable improvement in launch time relative to the baseline; in some runs, the measured time was slightly worse. Several reasons explain this outcome. First, the warm-up step is essentially the same kind of behavior already implicit in the baseline protocol, which itself involves repeated launches with short stabilization gaps; the system is therefore already partially warm at the moment of the measured launch, and an

additional brief launch does not create a materially new cache state. Second, the operational definition of cold launch used in this project – force-stopping the application and relaunching it – does not by itself guarantee an empty system state, because the kernel page cache, the ART/Dex cache, profile-guided compilation artifacts, and other runtime structures can persist across runs and even across reboots. Third, the additional churn introduced by the extra launch-and-kill cycle can occasionally interfere with the measured launch in ways that obscure any small benefit that might exist.

This first experiment is therefore best understood as a *negative result*. It is reported here because it sharpens the design question for the rest of the exploration: a generic, untargeted warm-up of the entire application is unlikely to help, because it does nothing that the baseline protocol is not already doing implicitly. To produce a meaningful effect, an intervention must either be *informed by observation* (so that it warms specifically the files or pages that matter to startup), or it must alter system conditions that the baseline protocol does not already alter (for example, scheduling priority during the launch window). The next three experiments pursue exactly these two paths.

3.5 Second Userspace Warm-Up

The second user-space experiment refines the warm-up idea by making it *observation-informed* rather than generic. Like the first attempt, it performs a short unmeasured launch of Gallery before the measured run; unlike the first attempt, it then inspects the live process to find out which files are actually open during startup, and uses that information to guide the actual warm-up step. Concretely, while the unmeasured launch is running, the monitor walks `/proc/<pid>/fd` for the Gallery process and records the set of file paths held open at that moment. Those paths are written to a small list file `warmup_files.txt`, which therefore captures a real, observed startup-file set without requiring a deeper kernel-side intervention.

Before each later *measured* launch, the monitor reads a small chunk from each file in that list. The intent of these pre-reads is to populate the Linux page cache with the corresponding file pages, so that when Gallery is subsequently launched for the measured run, some of the code, resource, or other startup-relevant data is already resident in memory and does not need to be fetched from storage. This is still a user-space-only intervention, but it is now guided by a concrete observation of what Gallery's startup actually opens, rather than by the implicit assumption that another full launch will warm the right state.

The two configurations are toggled at run time without rebuilding the source. The warm-up configuration is enabled by setting `EBPF_WARMUP=1` in the environment; the baseline configuration omits that variable. The two invocations used during measurement are:

Listing 1: Warm-up vs. baseline invocations of the monitor.

```
# Warm-up mode
adb shell "export LD_LIBRARY_PATH=/data/local/tmp:$LD_LIBRARY_PATH; \
           export EBPF_WARMUP=1; \
           /data/local/tmp/monitor_example"

# Baseline mode
adb shell "export LD_LIBRARY_PATH=/data/local/tmp:$LD_LIBRARY_PATH; \
           /data/local/tmp/monitor_example"
```

This second user-space warm-up is the first exploration attempt that produced a meaningful improvement in cold-launch time. Across the five measured runs, launch time was visibly lower than the baseline reference, while two of the other kernel-visible metrics remained close to baseline: total page allocations per run and number of distinct files touched per run did not change in a clearly informative way. As a result, the most useful single visualization for this experiment is the per-run launch-time comparison.

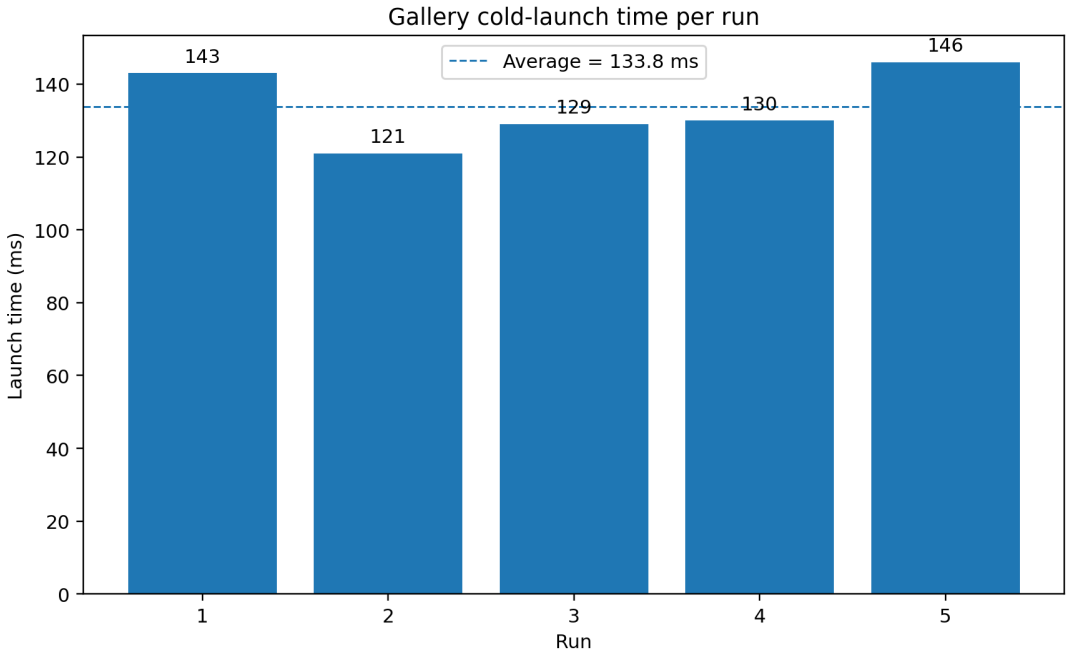

Figure 11: Gallery cold-launch time per run under the second user-space warm-up. The warm-up configuration produces a visible reduction in measured launch time relative to the baseline, while page allocation counts and files-touched counts remain close to baseline.

A subtle methodological issue must be acknowledged when interpreting this result. Re-running the baseline *after* a sequence of warm-up runs does not necessarily reproduce a truly cold initial state. The warm-up runs themselves populate the page cache, the ART/Dex cache, and the application's profile data, and several of those effects persist across runs and even across reboots. Deleting `warmup_files.txt` only removes the warm-up file list; it does not undo the cached pages or the compiled artifacts that earlier runs may have produced. Furthermore, the monitor unconditionally prints a "warm-up phase" log line whenever it starts a launch, while the warm-up logic itself only executes when `EBPF_WARMUP=1`. The presence of that log line in baseline runs is therefore not, by itself, evidence that warm-up logic was applied.

In practice, obtaining a cleaner cold baseline after warm-up experiments requires actively resetting the relevant system state, for example by dropping the page cache, resetting ART compilation for the application, clearing application data, or rebooting and exercising the system before the next measurement. The most reliable way to obtain a truly clean comparison, however, is to collect the baseline measurements *before* running any warm-up experiments, which is the order followed in this project.

3.6 Third Experiment: Kernel-Side Selective Startup Focus (Failed)

The third experiment is the first kernel-side attempt at improving repeated cold-launch behavior, implemented in `example.c`. The strategy is *launch-process locking with selective focus*: instead of tracing all matching activity broadly during the startup window, the eBPF program identifies the first process that begins startup file-open activity after the maps are reset and treats that process as the launch process. From that moment onward, tracing and accounting are restricted to that TGID. The intent is twofold: to keep the measurement focused on the actual Gallery startup path rather than on unrelated background work, and to reduce the kernel-side tracing overhead that might otherwise slightly perturb the launch.

In addition, this experiment introduces a small cap on startup file-open counting: after enough early startup opens have been recorded for the launch process, the eBPF program stops doing extra per-open accounting work. The underlying assumption is that the launch-critical file

activity happens early in the startup window, so continuing to count every later open contributes overhead without much corresponding benefit. Together, these two changes form a lightweight kernel-informed adjustment whose stated goal is to reduce monitoring interference and slightly improve measured launch time while preserving the existing experiment structure.

The measured outcome was modest at best for launch time, and dramatic but *misleading* for the other metrics. Under this kernel-side optimization, the average measured launch time over five runs was approximately 192 ms (per-run values: 191, 175, 184, 224, 190), compared with a baseline average of approximately 201 ms (per-run values: 204, 201, 194, 203, 203). The net effect is a difference of roughly 9 ms on average, which is within the normal run-to-run variability of the measurement and is therefore not a convincing improvement. By contrast, the page-allocation counts collapsed to roughly 320–322 per run (Run 1: 320, Run 2: 322, Run 3: 322, Run 4: 320, Run 5: 320), compared with baseline values of approximately 4691–4800 per run, which is approximately a fifteen-fold reduction. The files-touched count similarly dropped to between 0 and 1 entries, compared with 16–18 in the baseline.

The reason launch time barely changed is straightforward: this optimization does not materially alter the work the application actually performs during startup. It changes what the eBPF program observes and counts – it filters tracing to the launch TGID and caps open counting – but it does not change the application's real execution path. The expected effect on launch time is therefore limited to a small reduction in measurement overhead, which is consistent with the small, within-noise difference observed.

The reason the page-allocation counts collapse so dramatically is also straightforward, but is much more important to interpret correctly. In optimized mode, the eBPF programs only count events for the first TGID that performs a startup `openat/openat2` after the maps are reset. Every other allocation occurring elsewhere in the system during the launch window is filtered out. The baseline, by contrast, counts much more broadly across the system. The fifteen-fold "reduction" therefore reflects a change in the *observation scope*, not a real fifteen-fold reduction in the system's actual memory work. The same applies to the drop in files-touched counts: with tracing restricted to a single TGID and capped early, fewer file opens are *counted*, but this is not evidence that fewer file opens are *occurring*.

This experiment is honestly a failed optimization. It did not produce a convincing reduction in launch time, and the dramatic changes in the other metrics must not be misinterpreted as a system-wide improvement. The experiment is nonetheless informative because it makes a methodological point sharply: narrowing the instrumentation focus is not the same as reducing the underlying cost. To reduce the underlying cost, an intervention must change the launch's actual execution conditions, not merely what is being measured. The fourth experiment is constructed around exactly that distinction.

3.7 Fourth Experiment: Kernel-Guided Launch Priority Boost

The fourth experiment is the second kernel-side attempt and the strongest exploration result in this project. Its design is informed by the conclusion of the previous experiment: to reduce launch time meaningfully, the intervention must change the conditions under which the launch executes, not merely the scope of what is observed.

The mechanism is small, scoped, and explicit. The eBPF program (in `example.c`) captures the first TGID that performs an `openat/openat2` immediately after the maps are reset and records it as the launch process. The user-space monitor then reads that TGID and temporarily boosts its scheduling priority for the duration of the launch window using `renice -10`. The boost is applied only to the captured launch TGID and only while the launch is in progress; once the launch window closes, the priority returns to normal. The intervention is therefore kernel-informed – the target process is identified directly from kernel-side tracing rather than guessed from user-space timing – but the actual mechanism is a standard, well-understood scheduling adjustment.

Cold launch is sensitive to CPU scheduling and to short stalls during application initialization. Boosting the launch process at the moment it begins touching startup files reduces the contention it experiences against other runnable work and lets the launch-critical code progress more quickly. Because the boost is tightly scoped to the launch TGID and to the launch window, the intervention does not become a broad, system-wide priority change and does not require deep modification of Android internals.

Across the five measured runs under this configuration, the average measured launch time was 136 ms, with per-run values of 125, 137, 130, 148, and 140 ms. Compared with the baseline average of 203.4 ms, this corresponds to a substantial reduction in repeated cold-launch time for Gallery. The number of files touched during the launch window remained in a normal range (18–21), comparable to the baseline, indicating that the optimization did not fundamentally change the startup file path. Page allocations were higher under this configuration (approximately 8477–8721 per run); rather than indicating a regression, this is consistent with the launch process executing more aggressively when given more CPU time, because it progresses further through startup work within the launch window.

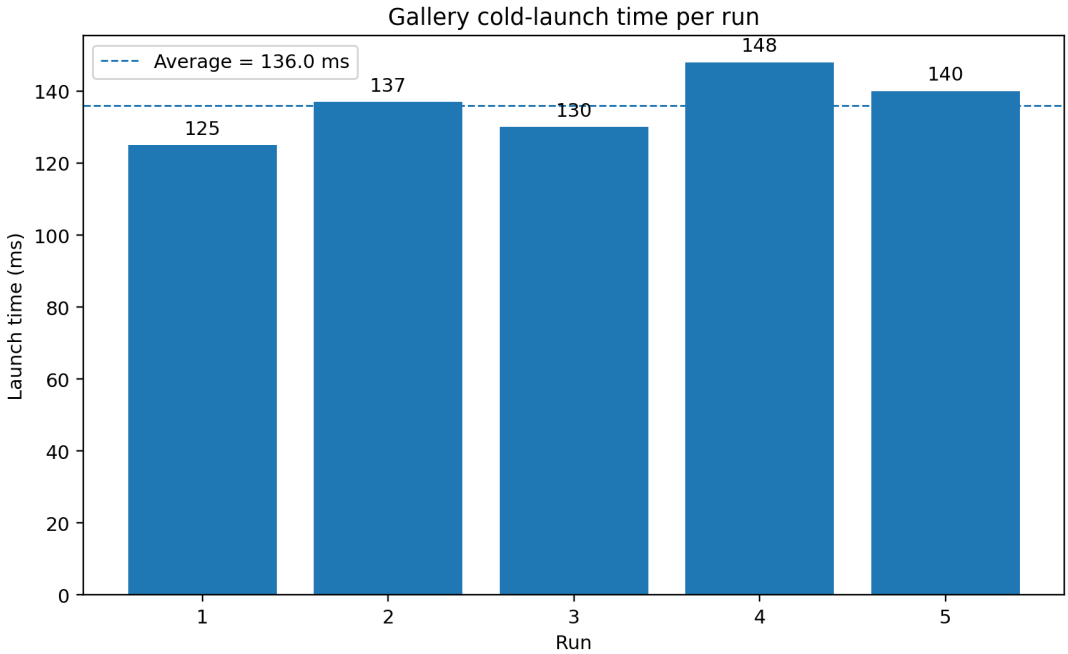

Figure 12: Gallery cold-launch time per run under the kernel-guided launch priority boost. The optimized configuration consistently produces lower launch times than the baseline, while file-touch counts remain in a comparable range and page allocation counts increase consistently with more aggressive startup progress under improved scheduling.

The result is credible because the intervention does not claim to alter Android internals broadly. It applies a small, targeted scheduling advantage to the specific process performing the launch, identified through kernel-side observation, and only for the brief window during which the launch is taking place. This experiment is the only kernel-side attempt that produced a clearly successful result, and it is a stronger exploration outcome than the third experiment because it changes actual launch execution conditions rather than mostly changing what is counted. The contribution should be read as a targeted, kernel-guided improvement in repeated cold-launch time for one well-chosen application, not as a general-purpose solution to application-launch latency.

4 Conclusion

This project used eBPF as a unifying observability framework to study three kernel-level dimensions of system behavior on an Android device – file I/O patterns, page allocation orders, and page allocation latency – and then used the resulting understanding to investigate whether eBPF-guided observation can help reduce repeated cold-launch overhead for the AOSP Gallery application. The characterization phase showed that kernel-visible behavior and user-visible behavior are not always aligned, that page allocations are dominated by small order-0 requests, and that allocation latency improves on average under heavier workloads while still admitting occasional expensive spikes. The exploration phase tested four lightweight optimization attempts against a controlled five-run baseline: a generic user-space warm-up that did not improve launch time, an observation-guided user-space warm-up that produced a measurable improvement, a kernel-side selective-tracing attempt whose dramatic apparent gains turned out to reflect a change in observation scope rather than a real performance improvement, and a kernel-guided launch-priority boost that produced a clear and defensible reduction in repeated cold-launch time. Taken together, the results support a careful, modest claim: eBPF observability can do more than describe the system; with care, it can also guide small, scoped interventions whose effects are real, measurable, and honestly bounded.

A Appendix: Build and Run Commands

This appendix documents the commands used during development on a Windows host with WSL, deployment to the Android device, and the runtime invocations used to collect baseline and optimized measurements. The runtime environment variables `EBPF_WARMUP` and `EBPF_Kernel_optimization` control which exploration mode the monitor runs in.

A.1 Compile `example.c` in WSL

```
cd ~/ebpf_project
cp /mnt/c/Users/Fathey/Downloads/OS_Project/eBPF/exploration/example.c .
make
# Output:
# example.o
```

A.2 Compile `monitor_example.cpp` in WSL

```
cp /mnt/c/Users/Fathey/Downloads/OS_Project/eBPF/exploration/monitor_example.cpp \
   ~/ebpf_project/jni/

~/android-ndk-r27d/ndk-build \
    NDK_PROJECT_PATH=. \
    NDK_APPLICATION_MK=Application.mk \
    APP_BUILD_SCRIPT=jni/Android.mk

# Outputs:
# ~/ebpf_project/libs/arm64-v8a/monitor_example
# ~/ebpf_project/libs/arm64-v8a/libc++_shared.so
```

A.3 Copy build artifacts back to Windows

```
cp example.o /mnt/c/Users/Fathey/Downloads/OS_Project/eBPF/exploration/
cp ~/ebpf_project/libs/arm64-v8a/monitor_example \
   /mnt/c/Users/Fathey/Downloads/OS_Project/eBPF/exploration/
```

```
cp ~/ebpf_project/libs/arm64-v8a/libc++_shared.so \
   /mnt/c/Users/Fathey/Downloads/OS_Project/eBPF/exploration/
```

A.4 Push all artifacts to Android

```
adb root

adb push C:\Users\Fathey\Downloads\OS_Project\eBPF\exploration\example.o /data/local/tmp/
adb push C:\Users\Fathey\Downloads\OS_Project\eBPF\exploration\monitor_example /data/local/
    tmp/
adb push C:\Users\Fathey\Downloads\OS_Project\eBPF\exploration\libc++_shared.so /data/local/
    tmp/
```

A.5 Prepare `monitor_example` on device

```
adb shell chmod +x /data/local/tmp/monitor_example
```

A.6 Deploy `example.o` to system BPF location

```
adb root
adb remount

adb shell cp /data/local/tmp/example.o /system/etc/bpf/
adb shell chmod 644 /system/etc/bpf/example.o
adb reboot
adb root
```

A.7 Verify BPF objects loaded

```
adb shell "ls /sys/fs/bpf | grep example"
```

A.8 Cleanup before a fresh baseline run

```
adb shell "rm /data/local/tmp/ebpf_results/warmup_files.txt"
adb shell "am force-stop com.android.gallery3d"
adb shell "pm clear com.android.gallery3d"
adb shell "cmd package compile --reset com.android.gallery3d"
adb shell "sync; echo 3 > /proc/sys/vm/drop_caches"
```

A.9 Run monitor – baseline cold launch

```
adb shell "export LD_LIBRARY_PATH=/data/local/tmp:$LD_LIBRARY_PATH; \
           /data/local/tmp/monitor_example"
# Use this for: Gallery App cold launch baseline
```

A.10 Run monitor – optimized mode

```
adb shell "export LD_LIBRARY_PATH=/data/local/tmp:$LD_LIBRARY_PATH; \
           export EBPF_Kernel_optimization=1; \
```

```
           /data/local/tmp/monitor_example"
# Use this for: Gallery App launch with kernel optimization enabled
```

A.11 Run monitor – userspace warm-up mode

```
adb shell "export LD_LIBRARY_PATH=/data/local/tmp:$LD_LIBRARY_PATH; \
           export EBPF_WARMUP=1; \
           /data/local/tmp/monitor_example"
# Use this for: Gallery App launch with userspace warm-up enabled
```

A.12 Pull result files to PC

```
adb pull /data/local/tmp/ebpf_results \
    "C:\Users\Fathey\Downloads\OS_Project\eBPF\exploration\ebpf_results"
```

A.13 Clear result directory for clean next experiment

```
adb shell "rm -rf /data/local/tmp/ebpf_results"
```

A.14 Final verification (debugging)

```
# Verify 5 programs attached
adb shell "ls /sys/fs/bpf/prog_example_*"

# Verify 5 maps pinned
adb shell "ls /sys/fs/bpf/map_example_*"
```

A.15 Quick rebuild reminders

If `example.c` is changed:

```
make -> push example.o -> copy to /system/etc/bpf/ -> reboot
```

If `monitor_example.cpp` is changed:

```
ndk-build -> copy from libs/arm64-v8a -> push monitor_example + libc++_shared.so -> run
```